

UCL DEPARTMENT OF COMPUTER SCIENCE

COMP0264: Generative Artificial Intelligence

Master Past Exam Revision Guide & Exhaustive Solutions

Compiling Examinations and Core Concepts for Cohorts 2025–2026

Prepared by **Antigravity AI** for UCL Computer Science

July 2026

Master Course Syllabus & Core Roadmap

This master revision guide compiles all examination papers and core theoretical foundations for UCL COMP0264 (Generative Artificial Intelligence), featuring **each question immediately followed by its exhaustive, step-by-step solution**:

Exam / Topic	Core Concepts and Key Formulations
Paper 1: Mock Variant A	Encoder/Decoder VAEs, Posterior Collapse, Transformers, LLMs, and Diffusion Models.
Paper 2: Mock Variant B	Generator/Discriminator VAEs vs GANs, Mode Collapse, Scaling Laws, and Alignment.
Appendix	Standard Formulas: Chain Rule ($N = 2, 3, \dots, n$), Bayes' Rule, ELBO, Gaussian KL, Softmax Scaling, DPO Loss, Score Matching SDEs.

PAPER 1: COMP0264 Mock Examination Variant A

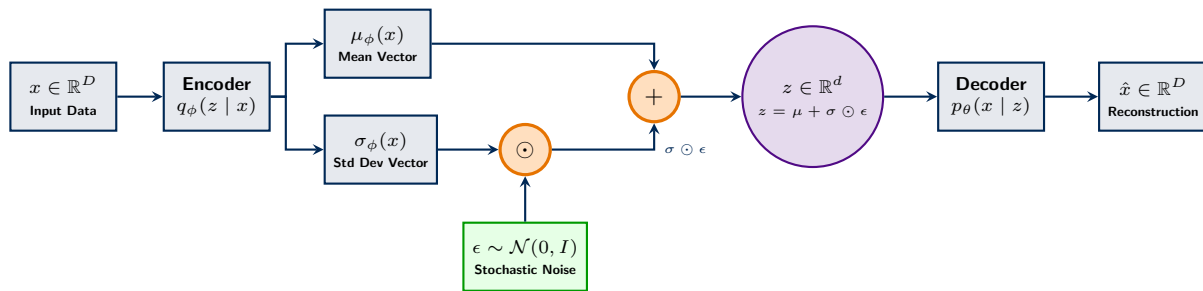
Question 1: Variational Autoencoders & Posterior Collapse [25 Marks]

- Describe the architecture of the encoder and decoder of a Variational Autoencoder (VAE). Discuss the training procedure including the loss function (ELBO) used for optimization. [12.5 Marks]
- Explain why training VAEs can be unstable and discuss the causes of the potential problem of posterior collapse (i.e. the situation in which the encoder's output becomes almost identical to the prior, regardless of the input). Compare this to mode collapse in GANs and discuss possible solutions. [12.5 Marks]

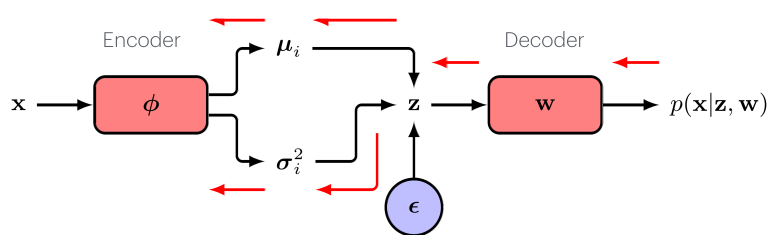
Solution 1: Variational Autoencoders & Posterior Collapse

Core Concept Summary: VAEs optimize the Evidence Lower Bound (ELBO) using an encoder $q_\phi(z | x)$ and decoder $p_\theta(x | z)$. The Reparameterization Trick ($z = \mu + \sigma \odot \epsilon$) enables end-to-end gradient backpropagation through stochastic latent bottlenecks.

Perspective 1: Vector Computational Graph & Reparameterization Trick (TikZ Block Diagram)



The Reparameterisation Trick



Generative AI 2025-26

Mirco Musolesi

Figure 1: *Official Lecture Diagram: VAE Architecture & Reparameterization Trick.*

Perspective 2: Official Lecture Slide Architecture (COMP0264 Lecture 4, Slide 36)

Exhaustive 5-Step Derivation of the Evidence Lower Bound (ELBO): Our objective is to maximize the marginal log-likelihood of observed data $\log p_\theta(x) = \log \int p_\theta(x, z) dz$, which is computationally intractable due to the integral over all high-dimensional latent states z .

Step 1: Introduce Variational Distribution $q_\phi(z | x)$: Multiply and divide the integrand inside the logarithm by the encoder's variational posterior $q_\phi(z | x)$:

$$\log p_\theta(x) = \log \int q_\phi(z | x) \frac{p_\theta(x, z)}{q_\phi(z | x)} dz = \log \mathbb{E}_{q_\phi(z|x)} \left[\frac{p_\theta(x, z)}{q_\phi(z | x)} \right]$$

Step 2: Apply Jensen's Inequality: Because the logarithm $\log(\cdot)$ is a strictly concave function, pushing the expectation outside the logarithm yields a strict lower bound ($\log \mathbb{E}[Y] \geq \mathbb{E}[\log Y]$):

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z | x)} \right] \equiv \mathcal{L}_{\text{ELBO}}(\theta, \phi)$$

Step 3: Algebraic Expansion into Reconstruction & Regularization Terms: Factorize the joint distribution $p_\theta(x, z) = p_\theta(x | z)p(z)$ and split the logarithm ($\log \frac{A \cdot B}{C} = \log A + \log \frac{B}{C}$):

$$\begin{aligned} \mathcal{L}_{\text{ELBO}}(\theta, \phi) &= \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x | z)p(z)}{q_\phi(z | x)} \right] \\ &= \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] + \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p(z)}{q_\phi(z | x)} \right] \\ &= \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] - \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{q_\phi(z | x)}{p(z)} \right] \\ &= \underbrace{\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)]}_{\text{Reconstruction Loss (Decoder)}} - \underbrace{\mathbb{E}_{q_\phi(z|x)} \left[\log \frac{q_\phi(z | x)}{p(z)} \right]}_{\text{KL Divergence Regularization (Encoder)}} \end{aligned}$$

Step 4: Exact Identity & KL Divergence Tightness: The exact gap between marginal log-likelihood $\log p_\theta(x)$ and the lower bound $\mathcal{L}_{\text{ELBO}}$ is the KL divergence between variational posterior $q_\phi(z | x)$ and true posterior $p_\theta(z | x)$:

$$\log p_\theta(x) = \mathcal{L}_{\text{ELBO}}(\theta, \phi) + KL(q_\phi(z | x) \parallel p_\theta(z | x))$$

Since $KL(q \parallel p) \geq 0$, maximizing $\mathcal{L}_{\text{ELBO}}(\theta, \phi)$ directly minimizes the KL divergence to the true posterior!

Step 5: Final Neural Network Training Loss: In neural network gradient optimization, we minimize the negative ELBO ($\mathcal{L}_{\text{VAE}} = -\mathcal{L}_{\text{ELBO}}$):

$$\mathcal{L}_{\text{VAE}}(\theta, \phi) = -\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] + KL(q_\phi(z | x) \parallel p(z))$$

- Encoder/Decoder Architecture & Reparameterization Trick:** A VAE parameterizes encoder $q_\phi(z | x)$ and decoder $p_\theta(x | z)$. The Reparameterization Trick sets $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$, allowing backpropagation $\nabla_\phi \mathcal{L}_{\text{ELBO}}$ via standard automatic differentiation.
- Posterior Collapse in VAEs vs Mode Collapse in GANs:**
 - Posterior Collapse in VAEs:** Occurs when an overly expressive autoregressive decoder ignores latent variable z , causing $q_\phi(z | x) \rightarrow p(z)$ ($KL \rightarrow 0$). Solved via β -annealing, KL thresholding (free bits), or VQ-VAE.
 - Mode Collapse in GANs:** Occurs when generator G collapses to generating samples from only a single mode. Solved via WGAN-GP, spectral normalization, or unrolled GANs.

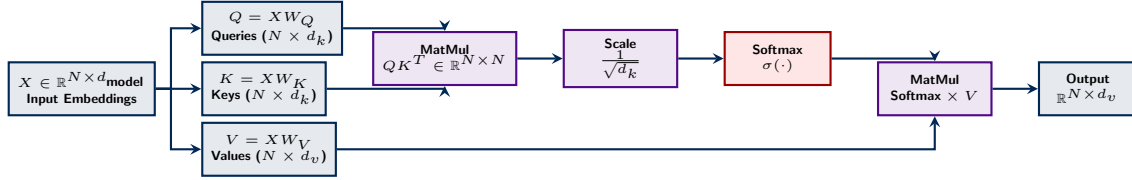
Question 2: Transformer Architectures & Scaled Self-Attention [25 Marks]

- Explain the self-attention mechanism in transformer architectures and why it is effective compared to recurrent neural networks. [5 Marks]
- Why is positional encoding necessary in transformers? Describe one method for incorporating positional information (e.g. sinusoidal or RoPE). [10 Marks]
- Discuss why we need scaled self-attention. Provide a mathematical justification by deriving the variance scaling factor $\sqrt{d_k}$. [10 Marks]

Solution 2: Transformer Architectures & Scaled Self-Attention

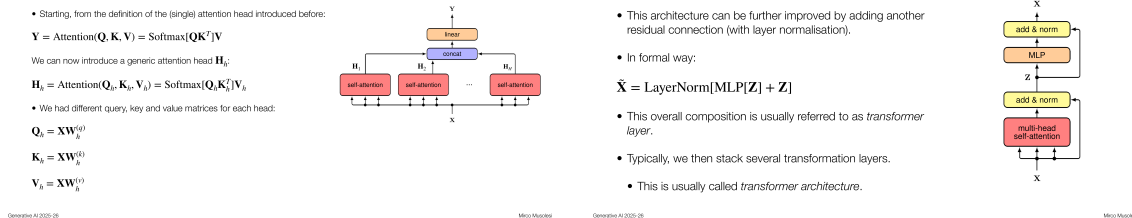
Core Concept Summary: Self-Attention computes pairwise token interaction matrices $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ in $O(1)$ path length. Scaling by $\sqrt{d_k}$ rescales dot-product variance back to 1, preventing softmax gradient saturation.

Perspective 1: Tensor Signal-Flow Diagram (TikZ Block Diagram)



Multi-Head Attention

Transformer Layer and Transformer Architecture



(a) Multi-Head Self-Attention (Slide 31)

(b) Full Transformer Layer (Slide 42)

Figure 2: Official Lecture Diagrams: Multi-Head Self-Attention and Full Transformer Architecture.

Perspective 2: Official Lecture Multi-Head Self-Attention & Layer Architecture (COMP0264 Lecture 2, Slides 31 & 42)

- (a) **Self-Attention Mechanism vs RNNs:** The Self-Attention mechanism maps a sequence of N tokens to contextual representations via $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right)\mathbf{V}$.

Formal Transformer Dimension Table (Color-Coded Numerical Example): Consider a fully consistent architecture: **Sequence Length $N = 512$ tokens**, **Hidden Embedding Size $d = 4096$** , **Heads $h = 32$** , **Projection Size $d_k = 128$** :

Concrete 3-Token Worked Numerical Calculation ($N = 3, d = 4, d_k = 2$): Let an input prompt contain $N = 3$ tokens: $\mathbf{x}_1 = \text{"The"}$, $\mathbf{x}_2 = \text{"cat"}$, $\mathbf{x}_3 = \text{"sat"}$ with embedding matrix $\mathbf{X} \in \mathbb{R}^{3 \times 4}$:

$$\mathbf{X} = \begin{pmatrix} 1.0 & 0.0 & 0.5 & 0.0 \\ 0.0 & 2.0 & 0.0 & 1.0 \\ 0.5 & 1.0 & 0.0 & 0.5 \end{pmatrix}, \quad \mathbf{W}_Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 1 & 0 \end{pmatrix}, \quad \mathbf{W}_K = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$$

1. Queries and Keys Calculation:

$$\mathbf{Q} = \mathbf{XW}_Q = \begin{pmatrix} 1.0 & 0.0 \\ 1.0 & 2.0 \\ 1.0 & 1.0 \end{pmatrix} \begin{matrix} \mathbf{q}_1 (\text{"The"}) \\ \mathbf{q}_2 (\text{"cat"}) \\ \mathbf{q}_3 (\text{"sat"}) \end{matrix}, \quad \mathbf{K} = \mathbf{XW}_K = \begin{pmatrix} 1.0 & 1.0 \\ 0.0 & 2.0 \\ 0.5 & 1.5 \end{pmatrix} \begin{matrix} \mathbf{k}_1 (\text{"The"}) \\ \mathbf{k}_2 (\text{"cat"}) \\ \mathbf{k}_3 (\text{"sat"}) \end{matrix}$$

2. Dot Products $\mathbf{QK}^T$ and Scaling ($\sqrt{d_k} = \sqrt{2} \approx 1.414$):

$$\mathbf{QK}^T = \begin{pmatrix} 1.0 & 0.0 \\ 1.0 & 2.0 \\ 1.0 & 1.0 \end{pmatrix} \begin{pmatrix} 1.0 & 0.0 & 0.5 \\ 1.0 & 2.0 & 1.5 \end{pmatrix} = \begin{pmatrix} 1.0 & 0.0 & 0.5 \\ 3.0 & 4.0 & 3.5 \\ 2.0 & 2.0 & 2.0 \end{pmatrix}$$

Scaling by $\frac{1}{\sqrt{2}} \approx 0.707 \implies \mathbf{S} = \begin{pmatrix} 0.707 & 0.000 & 0.354 \\ 2.121 & 2.828 & 2.475 \\ 1.414 & 1.414 & 1.414 \end{pmatrix}$. 3. **Softmax Attention Weights**

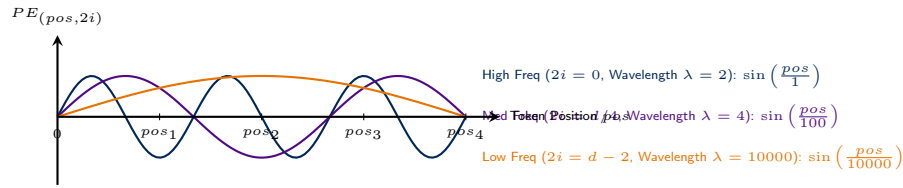
$\mathbf{A} = \text{softmax}(\mathbf{S})$: Row 2 ("cat"): $\text{softmax}(2.121, 2.828, 2.475) = (0.23 \ 0.49 \ 0.28)$. Token 2 ("cat") assigns 49% of its attention weight to itself, 28% to "sat", and 23% to "The"!

Unlike RNNs which suffer from $O(N)$ sequential bottleneck, Self-Attention computes this $N \times N$ interaction matrix in a single parallel GPU operation with $O(1)$ path length.

- (b) **Positional Encodings (Sinusoidal & RoPE)**: Because matrix multiplication $\mathbf{QK}^T$ is permutation-equivariant (ignoring token order), positional encodings inject sequence order into the model.

1. Absolute Sinusoidal Positional Encoding (Vaswani et al., 2017): For token position $pos \in \{0, 1, \dots, N-1\}$ and dimension index $i \in \{0, \dots, d/2-1\}$, the positional encoding vector $PE_{pos} \in \mathbb{R}^d$ is added directly to input embeddings ($X_{pos} \leftarrow X_{pos} + PE_{pos}$):

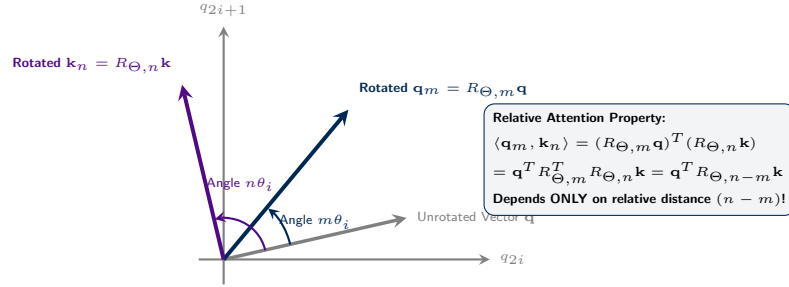
$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$



2. Rotary Position Embedding (RoPE - Su et al., 2021): Instead of adding position vectors to embeddings, RoPE multiplies Query/Key vectors by a 2D rotation matrix $R_{\Theta, m}^{(i)}$ for each 2D sub-vector pair (q_{2i}, q_{2i+1}) at token position m :

$$R_{\Theta, m}^{(i)} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix}, \quad \theta_i = 10000^{-2(i-1)/d}$$

$$\mathbf{q}_m^{(i)} = R_{\Theta, m}^{(i)} \mathbf{q}^{(i)}, \quad \mathbf{k}_n^{(i)} = R_{\Theta, n}^{(i)} \mathbf{k}^{(i)}$$



- (c) **Variance Scaling Derivation ($\sqrt{d_k}$)**: For independent zero-mean unit-variance Query/Key components $q_i, k_i \sim \mathcal{N}(0, 1)$:

$$\text{Var}(\mathbf{q} \cdot \mathbf{k}) = \text{Var}\left(\sum_{i=1}^{d_k} q_i k_i\right) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = \sum_{i=1}^{d_k} \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = \sum_{i=1}^{d_k} 1 = d_k$$

Thus $\text{StdDev}(\mathbf{q} \cdot \mathbf{k}) = \sqrt{d_k}$. Dividing by $\sqrt{d_k}$ rescales variance $\text{Var}\left(\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}}\right) = 1$, preventing extremely large dot products that would cause $\text{softmax}(\cdot)$ to saturate into one-hot vectors with near-zero gradients.

Question 3: Large Language Models (LLMs) [25 Marks]

- (a) Despite being trained with a next-token prediction objective, LLMs exhibit emergent behaviors such as in-context learning (ICL). Provide a critical explanation, grounded in probabilistic modeling, of why such behavior arises. [5 Marks]

- (b) In transformer-based LLMs, the conditional distribution $p(x_t | x_{<t})$ is parameterized using masked self-attention. Explain how masking ensures the autoregressive property is preserved during training. [5 Marks]
- (c) Suppose an LLM assigns high probability to fluent but factually incorrect sequences. Explain why maximum likelihood training does not guarantee factual correctness, and discuss one potential solution (e.g. RLHF/DPO or RAG). [7.5 Marks]
- (d) Explain how fine-tuning can lead to catastrophic forgetting from a probabilistic perspective. Use the Kullback-Leibler (KL) divergence to formalize the intuition. [7.5 Marks]

Solution 3: Large Language Models (LLMs) & Autoregressive Transformers

Core Concept Summary: In-Context Learning (ICL) acts as implicit Bayesian inference over latent tasks. Causal masking ($M_{ij} = -\infty$ for $j > i$) enforces autoregressive factorization. Preference alignment (DPO/RLHF) fixes MLE mode-covering hallucinations.

- (a) **Emergent In-Context Learning (ICL) as Implicit Bayesian Inference:** Despite being trained solely on next-token prediction, LLMs perform In-Context Learning without updating model parameters ($\nabla_{\theta} = 0$). From a Bayesian perspective, prompt tokens $C = (x_1, y_1, x_2, y_2, \dots, x_k, y_k, x_{\text{query}})$ provide evidence to infer a latent task concept variable τ :

$$P(y_{\text{query}} | x_{\text{query}}, C) = \sum_{\tau} P(y_{\text{query}} | x_{\text{query}}, \tau) P(\tau | C)$$

where the posterior over tasks is updated via Bayes' rule $P(\tau | C) = \frac{P(C|\tau)P(\tau)}{\sum_{\tau'} P(C|\tau')P(\tau')}$. The transformer's attention layers act as an implicit optimizer, using context tokens to isolate the latent task τ .

- (b) **Causal Masking Matrix & Autoregressive Preservation:** To parameterize $p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t | x_{<t})$ in parallel during training, a causal mask matrix M_{ij} is added to scaled dot-product attention scores before softmax:

$$\mathbf{S}_{\text{masked}} = \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} + \mathbf{M}$$

Matrix Causal Mask Grid Diagram & Color-Coded Worked Numerical Example ($\mathbf{N} = 4$):

For sequence length $\mathbf{N} = 4$ tokens, $M_{ij} = 0$ for $j \leq i$ (allowed attention) and $M_{ij} = -\infty$ for $j > i$ (masked future tokens):

Causal Mask Matrix $\mathbf{M} \in \mathbb{R}^{4 \times 4}$

	Tok 1	Tok 2	Tok 3	Tok 4
Token 1	0	$-\infty$	$-\infty$	$-\infty$
Token 2	0	0	$-\infty$	$-\infty$
Token 3	0	0	0	$-\infty$
Token 4	0	0	0	0

Green (0): Allowed past/present attention
 Red ($-\infty$): Masked out ($\exp(-\infty) \rightarrow 0.000$)
 Strictly enforces $p(x_t | x_{<t})$ autoregression

Because $\exp(-\infty) = 0$, softmax converts all red entries to exactly **0.000** attention weight. Thus token t cannot attend to any future token $j > t$, preserving strictly valid probability factorization during parallel training.

- (c) **Factual Correctness vs Maximum Likelihood Estimation (MLE) & DPO Solution:** MLE training minimizes reverse KL divergence $\mathcal{L}_{\text{MLE}} = KL(P_{\text{data}} \| P_{\theta}) = \mathbb{E}_{x \sim P_{\text{data}}} [-\log P_{\theta}(x)]$. Because reverse KL is **mode-covering**, model P_{θ} allocates non-zero probability to plausible-sounding but factually incorrect sequences (hallucinations).

Solution via Direct Preference Optimization (DPO): DPO optimizes model parameters θ directly on preference pairs (x, y_w, y_l) (preferred y_w vs dispreferred y_l) relative to a reference model π_{ref} :

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

This penalizes ungrounded mode-covering hallucinations without requiring a complex RL reward model.

- (d) **Probabilistic Formalization of Catastrophic Forgetting:** When fine-tuning a pre-trained model θ_{pre} on a narrow dataset D_{ft} , likelihood optimization shifts the model distribution $P_{\theta_{\text{ft}}}$, causing the KL divergence $KL(P_{\theta_{\text{pre}}} \parallel P_{\theta_{\text{ft}}}) \rightarrow \infty$. To prevent catastrophic forgetting of general reasoning capabilities, we add an explicit KL penalty to the fine-tuning loss:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{FT}}(\theta) + \lambda \cdot \mathbb{E}_{x \sim D_{\text{pre}}} [KL(P_{\theta_{\text{pre}}}(\cdot | x) \parallel P_{\theta}(\cdot | x))]$$

This anchors fine-tuned policy P_{θ} close to pre-trained base distribution $P_{\theta_{\text{pre}}}$.

Question 4: Diffusion Models [25 Marks]

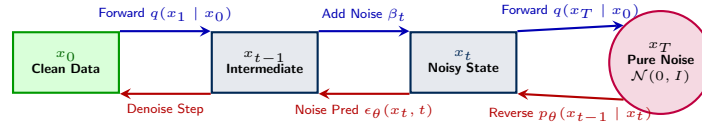
- (a) Describe the forward and reverse processes in a diffusion model. What is the goal of each process? Formulate the underlying mathematical model for both processes and explain how the reverse process is derived from the forward process. [12.5 Marks]
- (b) How is the reverse decoder in a diffusion model trained? What computational challenges arise in doing so, and what practical approach (ϵ -prediction MSE objective) is used instead? [12.5 Marks]

Solution 4: Diffusion Models & Score-Based Generative Frameworks

Core Concept Summary: Diffusion models define a forward noising chain $q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$ and train a reverse network $\epsilon_{\theta}(x_t, t)$ to predict added noise via simplified MSE loss.

- (a) **Forward Noising Chain and Reverse Denoising Process:**

Markov Chain Pipeline Diagram (TikZ Diagram):



Mathematical Derivation of Direct Closed-Form Forward Sampling: 1. **Single Noising Step:** For variance schedule $\beta_t \in (0, 1)$, set $\alpha_t = 1 - \beta_t$:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t)I) \implies x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_{t-1}$$

2. **Recursive Substitution back to Clean Data** x_0 : Substituting $x_{t-1} = \sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-2}$ into x_t yields:

$$x_t = \sqrt{\alpha_t \alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}}\bar{\epsilon}$$

3. **Closed-Form Forward Distribution:** Defining $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$:

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I) \implies x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Reverse Process Derivation : The reverse process $p_{\theta}(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$ is intractable directly, but conditioned on x_0 , Bayes' rule yields tractable Gaussian posterior $q(x_{t-1} | x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$ with mean:

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t$$

- (b) **ϵ -Prediction Objective & Training Simplification:** Substituting $x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}}$ into the posterior mean $\tilde{\mu}_t$ expresses target mean solely in terms of noise ϵ :

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right)$$

Minimizing the ELBO variational lower bound reduces to training neural network $\epsilon_\theta(x_t, t)$ to predict added Gaussian noise via the simplified Mean Squared Error (MSE) loss:

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t, x_0, \epsilon \sim \mathcal{N}(0, I)} \left[\left\| \epsilon - \epsilon_\theta \left(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|^2 \right]$$

PAPER 2: COMP0264 Mock Examination Variant B

Question 1: Generator/Discriminator GANs vs VAEs [25 Marks]

- (a) Formulate the GAN minimax game $V(G, D) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$. Prove that for a fixed Generator G , the optimal Discriminator is $D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$. [12.5 Marks]
- (b) Prove that when $D = D_G^*$, the minimax objective reduces to minimizing $2 \cdot JS(p_{\text{data}} \parallel p_g) - 2 \log 2$. Discuss mode collapse and WGAN-GP. [12.5 Marks]

Solution 1: Generator/Discriminator GANs vs VAEs

Core Concept Summary: Optimal GAN Discriminator $D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$ collapses the minimax game into minimizing Jensen-Shannon Divergence $JS(p_{\text{data}} \parallel p_g)$.

- (a) **Optimal Discriminator Derivation:** For a fixed G , objective $V(G, D) = \int [p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x))]$. Differentiating function $f(y) = a \log y + b \log(1 - y)$ with respect to $y = D(x)$ and setting to 0:

$$\frac{a}{y} - \frac{b}{1-y} = 0 \implies a(1-y) = by \implies y^* = \frac{a}{a+b}$$

Substituting $a = p_{\text{data}}(x)$ and $b = p_g(x)$ yields:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$

- (b) **Reduction to Jensen-Shannon Divergence:** Substituting $D_G^*(x)$ back into $V(G, D_G^*)$:

$$V(G, D_G^*) = \mathbb{E}_{p_{\text{data}}} \left[\log \frac{p_{\text{data}}(x)}{\frac{p_{\text{data}}(x) + p_g(x)}{2}} \right] + \mathbb{E}_{p_g} \left[\log \frac{p_g(x)}{\frac{p_{\text{data}}(x) + p_g(x)}{2}} \right]$$

$$V(G, D_G^*) = -\log 4 + KL \left(p_{\text{data}} \parallel \frac{p_{\text{data}} + p_g}{2} \right) + KL \left(p_g \parallel \frac{p_{\text{data}} + p_g}{2} \right) = 2 \cdot JS(p_{\text{data}} \parallel p_g) - 2 \log 2$$

Since $JS \geq 0$, minimum is achieved iff $p_g = p_{\text{data}}$. WGAN-GP replaces JS divergence with Earth Mover's distance using 1-Lipschitz gradient penalties.

Question 2: LLM Scaling Laws & Preference Alignment [25 Marks]

- (a) Explain the Chinchilla scaling laws (Kaplan et al., Hoffmann et al.) governing compute $C \approx 6ND$, model parameters N , and dataset tokens D . [12.5 Marks]
- (b) Derive the Direct Preference Optimization (DPO) loss from the Bradley-Terry preference model, explaining how DPO avoids training an explicit reward model. [12.5 Marks]

Solution 2: LLM Scaling Laws & Preference Alignment

Core Concept Summary: Chinchilla scaling dictates equal scaling of parameters N and tokens D ($N \propto \sqrt{C}$, $D \propto \sqrt{C}$). DPO reparameterizes reward $r(x, y) = \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$, enabling direct policy gradient optimization without a separate reward model.

- (a) **Chinchilla Scaling Laws:** Optimal compute allocation $C \approx 6ND$ requires scaling parameters N and tokens D in equal proportions: $N \propto C^{0.5}$, $D \propto C^{0.5}$ (20 tokens per parameter).
- (b) **DPO Loss Derivation:** Bradley-Terry model $P(y_w \succ y_l | x) = \sigma(r(x, y_w) - r(x, y_l))$. Closed-form optimal policy $\pi_r(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp \left(\frac{r(x, y)}{\beta} \right)$ implies $r(x, y) = \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$. Substituting $r(x, y)$ into Bradley-Terry cancels partition function $Z(x)$, yielding exact DPO loss:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

APPENDIX: Standard Formulas & Foundational Identities

1. Chain Rule of Probability: Step-by-Step Derivation

The Chain Rule permits factorizing any joint probability distribution over N random variables into a product of conditional probabilities.

Fundamental Definition (2 Variables, $N = 2$): For any two random variables x_1 and x_2 , the definition of conditional probability is:

$$p(x_2 | x_1) = \frac{p(x_1, x_2)}{p(x_1)} \implies p(x_1, x_2) = p(x_1) p(x_2 | x_1)$$

Complete Step-by-Step Derivation for 3 Variables ($N = 3$): We wish to derive the joint probability $p(x_1, x_2, x_3)$.

- Step 1 (First Factorization):** Treat the pair (x_1, x_2) as a single composite variable $Y = (x_1, x_2)$. Apply the 2-variable definition between Y and x_3 :

$$p(x_1, x_2, x_3) = p(Y, x_3) = p(Y) p(x_3 | Y) = p(x_1, x_2) p(x_3 | x_1, x_2)$$

- Step 2 (Second Factorization):** Apply the 2-variable definition to expand the marginal term $p(x_1, x_2)$:

$$p(x_1, x_2) = p(x_1) p(x_2 | x_1)$$

- Step 3 (Substitution & Final Equation):** Substitute $p(x_1, x_2) = p(x_1) p(x_2 | x_1)$ from Step 2 into the Step 1 equation:

$$\begin{aligned} p(x_1, x_2, x_3) &= [p(x_1) p(x_2 | x_1)] \cdot p(x_3 | x_1, x_2) \\ &= \mathbf{p}(\mathbf{x}_1) \mathbf{p}(\mathbf{x}_2 | \mathbf{x}_1) \mathbf{p}(\mathbf{x}_3 | \mathbf{x}_1, \mathbf{x}_2) \end{aligned}$$

General N Variables ($N \geq 2$): By repeating this substitution recursively for N variables:

$$p(x_1, x_2, \dots, x_N) = p(x_1) p(x_2 | x_1) p(x_3 | x_1, x_2) \cdots p(x_N | x_1, \dots, x_{N-1}) = p(x_1) \prod_{i=2}^N p(x_i | x_1, \dots, x_{i-1})$$

2. Conditional Probability Tables (CPTs) & Factor Representation

In a Directed Acyclic Graph (DAG) or Belief Network, every variable node x_i is governed by a **Conditional Probability Table (CPT)**, which specifies the probability distribution of x_i given all possible state configurations of its parent nodes $\text{pa}(x_i)$:

$$p(x_i | \text{pa}(x_i))$$

- Root Nodes** ($\text{pa}(x_i) = \emptyset$): The CPT reduces to the prior marginal probability distribution $p(x_i)$.
- Joint Factorization Rule:** The joint probability distribution over all N variables is computed by multiplying all node CPTs together:

$$p(x_1, x_2, \dots, x_N) = \prod_{i=1}^N p(x_i | \text{pa}(x_i))$$

- Junction Tree Assignment:** Every CPT factor $p(x_i | \text{pa}(x_i))$ must be assigned to exactly one clique C_k containing $\{x_i\} \cup \text{pa}(x_i)$, forming initial clique potential $\psi(C_k)$.

3. Bayes' Rule & Law of Total Probability

$$p(y | x) = \frac{p(x | y)p(y)}{p(x)} = \frac{p(x | y)p(y)}{\sum_{y'} p(x | y')p(y')}$$

3. Evidence Lower Bound (ELBO) & Jensen's Inequality

$$\log p(x) \geq \mathbb{E}_{q(z|x)} \left[\log \frac{p(x, z)}{q(z|x)} \right] = \mathbb{E}_{q(z|x)} [\log p(x|z)] - KL(q(z|x) \parallel p(z)) \equiv \mathcal{L}_{\text{ELBO}}$$

4. Gaussian Closed-Form KL Divergence

$$KL(q(z|x) \parallel p(z)) = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

5. Variance Scaling in Self-Attention ($\sqrt{d_k}$)

$$\text{Var}(q \cdot k) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k \implies \text{Var} \left(\frac{q \cdot k}{\sqrt{d_k}} \right) = \frac{d_k}{d_k} = 1$$

6. Direct Preference Optimization (DPO) Loss

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

7. Transformer Matrix Dimensions & Token Interpretations

For input sequence length N tokens and model embedding dimension d :

- N : Total number of tokens in the prompt sequence.
- d (d_{model}): Hidden embedding size per token (each token $\mathbf{x}_i \in \mathbb{R}^d$).
- d_k : Projection dimension of Queries and Keys per attention head ($d_k = d/h$).
- $\mathbf{Q} \in \mathbb{R}^{N \times d_k}$: Query matrix ($\mathbf{Q} = \mathbf{XW}_Q$). Row $\mathbf{q}_i$ is what token i searches for.
- $\mathbf{K} \in \mathbb{R}^{N \times d_k}$: Key matrix ($\mathbf{K} = \mathbf{XW}_K$). Row $\mathbf{k}_j$ is what token j offers.
- $\mathbf{V} \in \mathbb{R}^{N \times d_v}$: Value matrix ($\mathbf{V} = \mathbf{XW}_V$). Row $\mathbf{v}_j$ is token j 's information payload.
- $\mathbf{A} = \text{softmax} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \in \mathbb{R}^{N \times N}$: Pairwise token attention weights (A_{ij} = weight token i assigns to token j).
- $\mathbf{O} = \mathbf{AV} \in \mathbb{R}^{N \times d_v}$: Updated contextual output representation for each token i .

Symbol	Name	Shape (Color-Coded Example)	Interpretation in Terms of Tokens
N	Sequence Length	$N = 512$	Total number of tokens in the prompt sequence (512 tokens).
d (d_{model})	Embedding Size	$d = 4096$	Hidden vector dimension per token ($\mathbf{X} \in \mathbb{R}^{512 \times 4096}$ has 512 token rows $\mathbf{x}_i \in \mathbb{R}^{4096}$).
d_k	Head Projection	$d_k = 128$	Dimension of Query/Key vectors per head ($d_k = d/h = 4096/32 = 128$).
$\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$	Weights	4096×128	Projection matrices mapping 4096-dim embeddings to 128-dim head space.
$\mathbf{Q}$	Query Matrix	512×128	$\mathbf{Q} = \mathbf{XW}_Q$. Row $\mathbf{q}_i \in \mathbb{R}^{128}$ is what token i searches for .
$\mathbf{K}$	Key Matrix	512×128	$\mathbf{K} = \mathbf{XW}_K$. Row $\mathbf{k}_j \in \mathbb{R}^{128}$ is what token j offers to others .
$\mathbf{V}$	Value Matrix	512×128	$\mathbf{V} = \mathbf{XW}_V$. Row $\mathbf{v}_j \in \mathbb{R}^{128}$ is token j's content payload .
$\mathbf{S}$	Score Matrix	512×512	Scaled dot products $S_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{128}}$ between all 512×512 token pairs ($\sqrt{128} \approx 11.31$).
$\mathbf{A}$	Attention Weights	512×512	Softmax matrix $\mathbf{A} = \text{softmax}(\mathbf{S})$. Row i sums to 1.0, giving weights token i assigns to all 512 tokens.
$\mathbf{O}_{\text{head}}$	Head Output	512×128	Contextual output $\mathbf{O} = \mathbf{AV}$. Row $\mathbf{o}_i = \sum_{j=1}^{512} A_{ij} \mathbf{v}_j \in \mathbb{R}^{128}$ is updated contextual representation for token i .